

Was die Copilot CLI ist

- Der **Coding Agent in deinem Terminal**: liest Dateien, schreibt Code, führt Befehle aus – du bestätigst jeden Schritt.
- Gleiche Agent-Basis („Harness“) wie der Copilot Coding Agent in der Cloud. Was du lokal lernst, gilt auch für delegierte Tasks.
- GitHub eingebaut: Repos, Issues & PRs per natürlicher Sprache.
- Editor-unabhängig – funktioniert mit jedem Setup, auch remote/SSH.
- Erweiterbar über MCP-Server, Custom Agents & Skills.

Drei Stufen der Autonomie

Vorschlag	Copilot schlägt vor, du führst aus. Standard – und für die meiste Arbeit richtig.
Plan	<code>/plan</code> : Copilot legt den Weg offen, du gibst frei, dann setzt es um. Der Sweet Spot für größere Aufgaben.
Autopilot	Copilot arbeitet bis zum Ziel durch. Nur in Branch, Container oder Sandbox – und nur, wenn du das Ergebnis ohnehin reviewst.

`/delegate` schiebt einen Task in die Cloud: der Coding Agent öffnet Branch und Draft-PR. Das Review bleibt bei dir – der PR ist der Übergabepunkt.

Usage-Based Billing (seit 01.06.2026)

- **AI Credits statt Premium Requests**: abgerechnet wird die tatsächliche Modell-Arbeit – tokenbasiert, zu Modell-API-Preisen. Modell-Multiplikatoren gibt es nicht mehr.
- **1 Credit = 0,01 USD**. Budgets werden in USD gesetzt, Verbrauch in Credits angezeigt (10 \$ = 1.000 Credits).
- **Grundpreise unverändert**: Business 19 \$/Nutzer/Monat inkl. 19 \$ Credits · Enterprise 39 \$ inkl. 39 \$.
- **Kein Fallback-Modell mehr**: Verbrauch wird über Guthaben und Admin-Budgets gesteuert.

Aktionszeitraum – wichtig für die Budgetplanung:

Bestandskunden mit Business/Enterprise erhalten vom **01.06. bis 01.09.2026** ein deutlich erhöhtes inkludiertes Credit-Kontingent. Danach fällt es auf die Standardwerte zurück.

Verbrauchszahlen aus Juni bis August sind also **nicht** repräsentativ. Budgets und Alerts jetzt aufsetzen – nicht erst im September.

Die drei Token-Arten

Input	was du sendest: Prompt + Kontext – wächst mit großen Dateien
Output	was du bekommst: Antwort + Tool-Nutzung – pro Token am teuersten
Cache	wiederverwendeter Kontext – am günstigsten, macht Sessions effizient

Was kostet was?

- **Flatrate (keine Credits)**: Code Completions & Next Edit Suggestions – unbegrenzt in bezahlten Plänen.
- **Credits**: Chat · Edits · Agent Mode · Copilot CLI · Coding Agent · Spark & Spaces.
- **Credits + Actions-Minuten**: Copilot Code Review und der Coding Agent (agentische Infrastruktur).
- **Verbrauchstreiber**: Modellwahl, Prompt- & Antwortgröße, Kontext/Cache, agentische Workflows, MCP/Skills. Große Kontextfenster (bis 1 Mio. Tokens) sind mächtig – und teuer.

Pool & Budgets – so fließt das Geld

- **1. Pool zuerst**: Jede Lizenz zahlt ihr Credit-Guthaben in einen gemeinsamen Enterprise-Pool ein – alle Nutzer schöpfen zuerst daraus.
- **2. Flex-Allotment**: Ein variabler Zusatzanteil oberhalb der Base Credits, der automatisch greift und von GitHub angepasst werden kann.
- **3. Budget = Zusatzausgaben**: Erst nach Pool und Flex greift das Budget – es erlaubt und begrenzt Mehrverbrauch.
- **4. Ebenen**: Enterprise → Organisation → User – plus Cost Center für Teams. Ein User-Budget von 0 \$ blockiert sofort.
- **5. Alerts automatisch**: Warn-Mails an Admins bei 75 / 90 / 100 %.
- **6. Budget erreicht = Stopp**: Betroffene sind blockiert bis Aufstockung oder neuer Abrechnungszyklus.
- **7. Monitoring**: Die Billing-UI trennt inkludierte und zusätzliche Ausgaben – auch mitten im Zyklus. Nicht genutzte Credits verfallen zum Monatsende.

Die größten Kostenhebel

- Ein präziser Prompt spart eine ganze Iterationsrunde – der größte Hebel überhaupt.
- `/compact` und `/clear` statt endloser Sessions. Der billigste Kontext ist der, den du nicht sendest.
- Nicht jedes Problem braucht ein Frontier-Modell.
- Agent-Läufe mit `/plan` starten – ein falscher Plan kostet mehr als die Planprüfung.
- `/usage` zeigt den Verbrauch der laufenden Session.

Sicherheit & Governance

- **Jede Aktion lesen, verstehen, dann bestätigen**. Der Bestätigungsdialog ist kein Hindernis, er ist das Sicherheitsnetz.
- **Ordner-Trust bewusst vergeben** – nur dort, wo die CLI arbeiten soll.
- **Keine Secrets in Prompts** – auch nicht „nur zum Testen“.
- **Vor jedem Agent-Lauf committen** – dann bleibt Verwerfen gefahrlos.
- **Destruktive Befehle** (`rm`, `force-push`) doppelt prüfen. `--deny-tool` sticht immer `--allow-tool`.
- **Trust & Datenschutz**: Business/Enterprise-Prompts und -Code werden nicht zum Modelltraining verwendet. Org-Policies steuern Features, Modelle und Datenfluss – Details im Trust Center.
- **Review-Pflicht bleibt**: Copilot ist Assistent, nicht Autopilot. Der PR ist und bleibt der Kontrollpunkt.

Prompt-Praxis – und warum sie wirkt

Die sechs Punkte stammen aus dem *GitHub-Playbook*. Die Formel zum Abheften steht im *Cheat-Sheet* – hier steht, warum sie funktioniert.

1 · Klare Instruktionen

Persona vorgeben („agiere als Senior-Java-Engineer“), Format & Länge nennen, Prompt-Teile mit Trennern strukturieren, Schritte auflisten. Das Modell rate nicht – es füllt Lücken, und zwar mit dem Wahrscheinlichsten, nicht dem Gewünschten.

2 · Referenzen mitgeben

Quelltext, Doku oder Spezifikation anhängen und darauf verweisen – senkt selbstbewusst-falsche Antworten deutlich. Ohne Referenz erfindet das Modell eine plausible Antwort, und plausibel ist nicht richtig.

3 · Komplexes zerlegen

Große Aufgaben haben überproportional hohe Fehlerraten – der Fehler in Schritt 2 pflanzt sich durch Schritt 7 fort. Beauftrage kleine, prüfbare Teilschritte; fasse lange Dialoge zusammen.

4 · Denkzeit geben

Lass erst den Lösungsweg erklären, dann den Code schreiben. Das Modell „reflektiert“ und konvergiert zu besseren Antworten – und du siehst den Denkfehler, bevor er Code geworden ist.

5 · Externe Tools nutzen

Tests, Linter und Compiler prüfen zuverlässiger als das Modell sich selbst. „... und führ `mvn -q test` aus“ gehört in den Auftrag – dann korrigiert sich der Agent selbst, statt dir kaputten Code zu übergeben.

6 · Systematisch testen

Behandle Prompt-Änderungen wie Code-Änderungen: vergleichen, messen, behalten was wirkt. Was im Team funktioniert, gehört in die AGENTS.md – dort wirkt es für alle und liegt im Review-Prozess.

Erweiterbarkeit: MCP, Custom Agents & Skills

MCP-Server	verbinden Copilot mit euren Systemen – Jira, Confluence, Datenbanken, internen APIs. Verwaltung über <code>/mcp</code> .
Custom Agents	spezialisierte Agents mit eigenem Auftrag und eigenem Modell – z. B. ein Review-Agent nach euren Konventionen. Auswahl über <code>/agent</code> .
Skills	wiederverwendbare Fähigkeiten, die ein Agent bei Bedarf zieht – statt sie in jeden Prompt zu schreiben.

Alles drei liegt als Datei im Repo und damit im Review-Prozess. Das ist der Punkt: Team-Standards werden versioniert, nicht mündlich weitergegeben. `/env` zeigt, was in der Session gerade geladen ist.

IntelliJ oder CLI – wann was?

IDE	tippen & navigieren: Completions, Next Edit Suggestions, Inline-Chat beim Schreiben
CLI	beauftragen & delegieren: Repo-Aufgaben, Skripte, <code>/plan</code> , <code>/delegate</code>
Beides	gleiche Lizenz, gleiche Regeln – AGENTS.md wirkt überall

IntelliJ & JetBrains, Stand 2026: Completions & NES · Chat, Inline-Chat & Inline-Agent · Agent Mode inkl. Custom-, Sub- & Plan-Agents · MCP-Support mit Auto-Approve-Regeln · CLI-Integration & Unified-Sessions-Ansicht · Copilot als Agent im JetBrains AI Assistant.

Setup: Plugin „GitHub Copilot“ installieren → OAuth-Login. Für Preview-Features muss die Editor-Preview-Policy der Org aktiv sein.

Neue Features landen meist zuerst in VS Code – die JetBrains-Lücke ist 2026 klein geworden. Aktueller Stand: github.blog/changelog.

Deine erste Woche

Tag 1	CLI installieren, im echten Projekt starten, drei kleine Aufgaben stellen – bewusst welche, die du selbst beurteilen kannst.
Tag 2–3	AGENTS.md für ein Repo schreiben. Kurz halten: Stack, Konventionen, Test-Befehl, drei Don'ts.
Tag 4	Erster <code>/plan</code> -Lauf an einer echten Aufgabe. Plan lesen, korrigieren, dann laufen lassen.
Tag 5	<code>/usage</code> anschauen: Was hat die Woche gekostet, und wofür?

Fang mit Aufgaben an, deren Ergebnis du sicher beurteilen kannst. Vertrauen entsteht durch überprüfte Treffer, nicht durch Hoffnung.

Woran du merkst, dass es wirkt

- Du schreibst weniger Boilerplate – und merkst es nicht mal mehr.
- Tests entstehen früher, weil sie billiger geworden sind.
- Der erste Entwurf einer unbekannten Datei kostet Minuten statt einer Stunde.
- Reviews werden nicht länger – wenn doch, war der Prompt zu groß.

Warnsignal: Wenn ihr Code merged, den niemand erklären kann, ist nicht das Werkzeug das Problem, sondern der Prozess. Die Review-Pflicht ist kein Bürokratie-Rest – sie ist der Grund, warum das Ganze funktioniert.

Häufige Fragen

Ersetzt das Entwickler?	Nein. Es verschiebt Zeit von Tippen zu Denken und Prüfen. Die Verantwortung für den Code bleibt, wo sie war.
Landet unser Code bei GitHub?	Business/Enterprise-Prompts und -Code werden nicht zum Modelltraining verwendet. Details im Trust Center.
Was, wenn das Budget leer ist?	Betroffene Nutzer sind blockiert, bis aufgestockt wird oder der neue Zyklus startet. Kein Fallback-Modell mehr.
Welches Modell ist das beste?	Die falsche Frage. Ein präziser Prompt mit gutem Kontext schlägt das teurere Modell fast immer.

Glossar

Harness	die Agent-Maschinerie unter Copilot – dieselbe in CLI und Cloud
Agent	arbeitet mehrschrittig: liest, plant, ändert, prüft – statt nur zu antworten
Subagent	ein Agent, den ein Agent beauftragt – parallelisierbar via <code>/fleet</code>
Token	kleinste Abrechnungseinheit; Input, Output und Cache werden getrennt gezählt
AI Credit NES	GitHubs Verrechnungseinheit – 1 Credit = 0,01 USD Next Edit Suggestion – schlägt die nächste Änderung vor, nicht nur die nächste Zeile. Flatrate.
MCP	Model Context Protocol – Standard, über den Copilot externe Systeme anspricht
AGENTS.md	Dauer-Regeln fürs Repo – wirken in jeder Session und jedem Agent-Lauf

Die drei Merksätze

1. Copilot ist Assistent, nicht Autopilot – jedes Ergebnis wird gereviewt.
2. Kontext + präziser Prompt schlagen jedes Modell-Upgrade.
3. Verstehen, dann ausführen – besonders im Terminal.

Ressourcen

Doku	docs.github.com/copilot
Change Log	github.blog/changelog
Trust Center	copilot.github.com/trust.page

Fragen während und nach dem Workshop?

Ahmed Diab – a.diab@reply.de

Daniel Snell – d.snell@reply.de

Cluster Reply · Agentic DevOps · Das Übungs-Repo bleibt für dich verfügbar.